



FORMULARIO DE PRODUCCIÓN DE CONTENIDOS

NOMBRE DE LA ASIGNATURA: Desarrollo en plataformas
NOMBRE COMPLETO DEL DOCENTE CONTENIDISTA: Damián Aníbal Nicolalde Rodríguez

DESARROLLO

1. INTRODUCCIÓN DE LA CLASE (2 párrafos)

En el desarrollo web moderno, las aplicaciones ya no se limitan a mostrar contenido estático; deben reaccionar, adaptarse y responder en tiempo real a las acciones del usuario. En este contexto, la manipulación del DOM (Document Object Model) se convierte en la base de toda interactividad, permitiendo crear, modificar o eliminar elementos visuales de manera dinámica. Combinada con la gestión de eventos y las validaciones en tiempo real, esta capacidad dota a las interfaces de un comportamiento inteligente y sensible, en el que los datos ingresados se verifican al instante, se guardan localmente y se actualizan sin recargar la página.

Durante esta clase, el estudiante explorará cómo el DOM API, junto con herramientas modernas como la Fetch API, Web Storage (localStorage y sessionStorage), IndexedDB y los atributos ARIA, permiten construir experiencias web completas, seguras y accesibles. Estas tecnologías trabajan de forma integrada para lograr aplicaciones que consumen y almacenan datos de manera asíncrona, responden visualmente a las interacciones, mantienen persistencia entre sesiones y son inclusivas para todo tipo de usuarios. Con ello, el estudiante comprenderá cómo unir interactividad, persistencia y accesibilidad en un solo flujo de desarrollo coherente y profesional.

Clase 5: Manipulación del DOM y eventos

RDA 1: Programar aplicaciones web dinámicas, interactivas y con diseño visualmente adaptable, que permita mejorar la experiencia del usuario.

Reto # 1

5.1. Manipulación del DOM y eventos

El **DOM (Document Object Model)** es la interfaz que conecta el código JavaScript con la estructura visual del documento HTML. Es un modelo jerárquico que representa cada etiqueta, texto o atributo como un nodo dentro de un árbol de objetos. Comprender su funcionamiento permite modificar dinámicamente el contenido, el estilo y el comportamiento de una página sin necesidad de recargarla.



La manipulación del DOM constituye el corazón del desarrollo web interactivo: a través de JavaScript, los programadores pueden crear, eliminar, mover o cambiar elementos del documento, reaccionar ante las acciones del usuario mediante eventos y generar experiencias adaptables y accesibles. En este tema, se estudiarán las técnicas modernas para recorrer y modificar el DOM, los tipos de eventos más comunes y los patrones de buenas prácticas que permiten mantener un código limpio y eficiente.

1. Estructura del DOM: el árbol del documento

Cuando el navegador carga una página, convierte el HTML en una estructura de objetos que puede manipularse mediante JavaScript. Cada etiqueta se transforma en un nodo y todos los nodos conforman un árbol DOM.

Ejemplo de código HTML:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Bienvenido</h1>
    <p id="mensaje">Hola desde el DOM</p>
  </body>
</html>
```

Estructura en árbol:

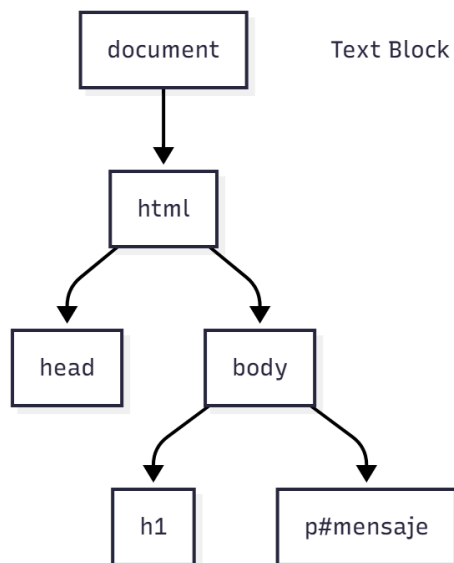


Imagen 1. Representación del árbol DOM.

Fuente: Elaboración propia (2025)

Cada elemento puede accederse, modificarse o eliminarse mediante la DOM API de JavaScript.



2. Selección de elementos

La forma más común de acceder a los elementos del DOM es usando selectores CSS. Desde ES6, las funciones recomendadas son `querySelector()` y `querySelectorAll()`.

```
// Seleccionar un único elemento
const titulo = document.querySelector("h1");

// Seleccionar por id
const parrafo = document.querySelector("#mensaje");

// Seleccionar múltiples elementos
const botones = document.querySelectorAll("button");
```

Tabla 1. Funciones para acceder a los elementos del DOM

Método	Descripción	Retorna
<code>getElementById(id)</code>	Busca por ID único	Elemento
<code>querySelector(sel)</code>	Primer elemento que cumple el selector	Elemento
<code>querySelectorAll(sel)</code>	Todos los elementos que cumplen el selector	NodeList
<code>getElementsByClassName(clase)</code>	Por clase CSS	HTMLCollection

Fuente: Elaboración propia (2025)

3. Creación y modificación de nodos

El DOM permite crear y agregar elementos nuevos dinámicamente.

```
const nuevo = document.createElement("div");
nuevo.textContent = "Elemento generado desde JS";
document.body.appendChild(nuevo);
```

También pueden modificarse atributos o estilos:

```
nuevo.setAttribute("class", "alerta");
nuevo.style.color = "red";
```

Eliminar o reemplazar nodos:

```
const viejo = document.getElementById("mensaje");
viejo.remove(); // elimina
```

4. Eventos: interacción con el usuario

Los eventos son acciones detectadas por el navegador: clics, movimientos, pulsaciones o cargas.

Para responder a ellos, se utilizan manejadores de eventos o *event listeners*.

```
const boton = document.querySelector("button");
boton.addEventListener("click", () => {
  alert("Has hecho clic en el botón");
});
```

Tabla 2. Tipos comunes de eventos:

Categoría	Evento	Descripción
Ratón	click, dblclick, mouseenter, mouseleave	Interacción con el mouse
Teclado	keydown, keyup, keypress	Pulsaciones de teclas
Formulario	submit, focus, input, change	Interacción con campos de entrada
Documento	DOMContentLoaded, scroll, resize	Cambios globales del documento

Fuente: Elaboración propia (2025)

5. Propagación y delegación de eventos

Cuando ocurre un evento, este se propaga en tres fases:

1. **Captura:** desde la raíz hasta el elemento objetivo.
2. **Objetivo:** se ejecuta el evento en el elemento.
3. **Burbujeo:** vuelve hacia arriba por la jerarquía.

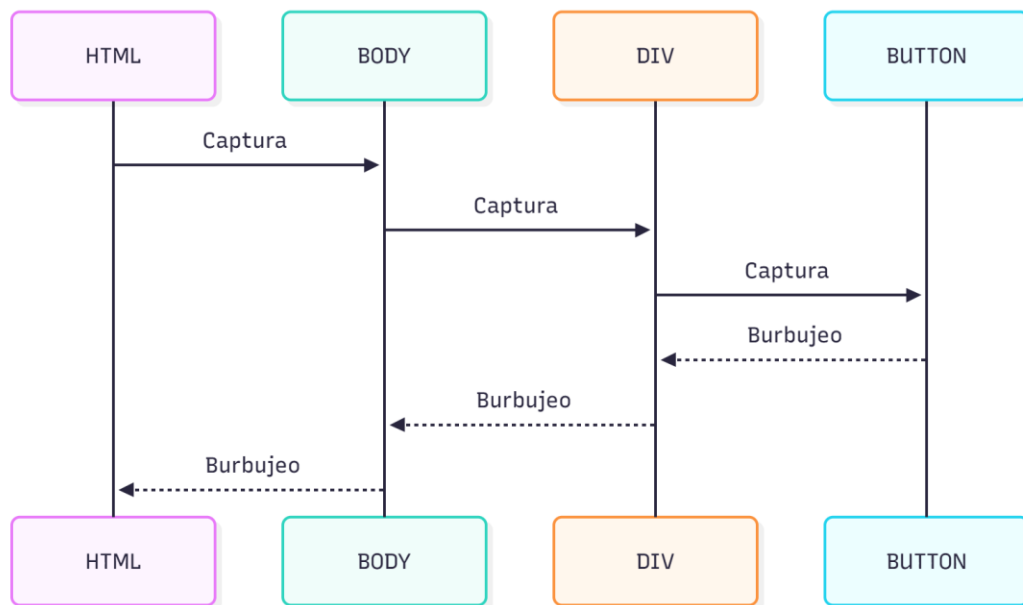


Imagen 2. Flujo de propagación de eventos (captura y burbujeo).

Fuente: Elaboración propia (2025)



Para optimizar, se puede usar delegación de eventos, escuchando el evento en un elemento contenedor y detectando cuál hijo lo originó:

```
document.querySelector("#lista").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    console.log("Se hizo clic en:", e.target.textContent);
  }
});
```

6. Eventos modernos y asincronía

Los eventos también pueden combinarse con operaciones asíncronas, por ejemplo, para cargar datos al hacer clic.

```
document.querySelector("#cargar").addEventListener("click", async () => {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts/1");
  const data = await res.json();
  console.log("Datos cargados:", data);
});
```

Aquí, el evento `click` se convierte en el disparador de una Promesa resuelta con `await`.

7. Buenas prácticas en manipulación del DOM

Tabla 3. Manipulación del DOM (buenas prácticas)

Práctica	Recomendación	Beneficio
Usar selectores claros	<code>querySelector</code> y <code>querySelectorAll</code>	Código más legible
Evitar modificaciones en bucle	Usar fragmentos (<code>DocumentFragment</code>)	Mejora de rendimiento
Delegar eventos	Escuchar en contenedores	Menor consumo de recursos
Separar lógica y presentación	No mezclar <code>onclick</code> en HTML	Código modular
Usar <code>addEventListener</code>	Permite múltiples manejadores	Mayor flexibilidad



8. Diagrama de interacción DOM y eventos

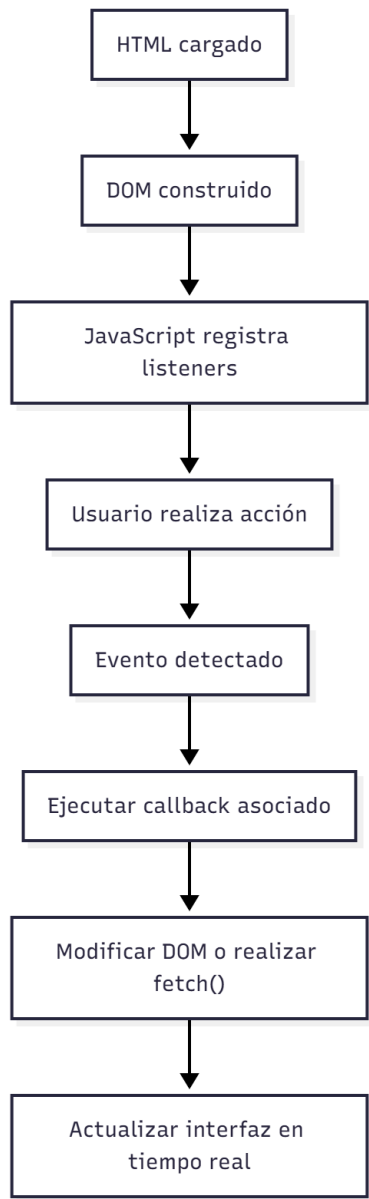


Imagen 3. Flujo de interacción entre el DOM, eventos y lógica JavaScript.
Fuente: Elaboración propia (2025)



9. Ejemplo integrador: lista interactiva dinámica

```
<ul id="tareass"></ul>
<input id="nuevaTarea" placeholder="Escribe una tarea..." />
<button id="agregar">Agregar</button>

<script>
  const lista = document.querySelector("#tareass");
  const input = document.querySelector("#nuevaTarea");
  const boton = document.querySelector("#agregar");

  boton.addEventListener("click", () => {
    const texto = input.value.trim();
    if (!texto) return;
    const li = document.createElement("li");
    li.textContent = texto;
    li.addEventListener("click", () => li.remove());
    lista.appendChild(li);
    input.value = "";
  });
</script>
```

Resultado:

Cada clic en “Agregar” añade una nueva tarea.

Cada clic sobre una tarea la elimina dinámicamente.

10. Conclusión

La manipulación del DOM y el manejo de eventos constituyen la base de toda interacción en el desarrollo web moderno. Comprender cómo JavaScript modifica el documento en tiempo real y cómo los eventos conectan al usuario con la lógica del sistema permite crear interfaces interactivas, reactivas y accesibles.

5.2. DOM API y validación dinámica

El DOM API (Document Object Model Application Programming Interface) es el conjunto de métodos y propiedades que JavaScript proporciona para acceder, modificar y validar dinámicamente los elementos de una página web.

Mientras en el tema anterior se abordó la manipulación del DOM y los eventos, en este apartado se explora cómo aplicar esas mismas técnicas para crear validaciones automáticas, reactivas y persistentes, combinando las interfaces del DOM con el almacenamiento del navegador (Web Storage API).

El objetivo es que el estudiante aprenda a detectar errores en tiempo real, mostrar retroalimentación visual inmediata, y conservar temporalmente los datos ingresados, incluso cuando el usuario recarga la página. Con ello, se fortalece el concepto de interactividad y experiencia de usuario (UX), al transformar la página estática en un sistema inteligente y accesible.



1. El rol del DOM API en la validación

El **DOM API** permite escuchar y reaccionar ante los cambios en los elementos de un formulario.

Cada campo (`input`, `select`, `textarea`) genera eventos (`input`, `change`, `blur`, `focus`) que pueden interceptarse para aplicar reglas de validación, sin depender del envío del formulario.

Ejemplo básico:

```
<form id="registro">
  <input id="nombre" placeholder="Nombre completo" />
  <small id="errNombre" class="error"></small>
</form>

<script>
  const nombre = document.querySelector("#nombre");
  const errNombre = document.querySelector("#errNombre");

  nombre.addEventListener("input", () => {
    if (nombre.value.trim().length < 3) {
      errNombre.textContent = "El nombre debe tener al menos 3 caracteres.";
      nombre.classList.add("invalido");
    } else {
      errNombre.textContent = "";
      nombre.classList.remove("invalido");
    }
  });
</script>
```

Comportamiento: Cada vez que el usuario escribe, el sistema evalúa en tiempo real si cumple la regla. No se necesita recargar la página ni presionar un botón.

2. API nativa de validación HTML5

El navegador ya incluye un mecanismo de validación nativa que puede controlarse mediante JavaScript usando propiedades del DOM:

Tabla 4. Propiedades del DOM para validación nativa

Propiedad / Método	Descripción	Ejemplo
input.validity	Devuelve un objeto con el estado de validación	if (campo.validity.valueMissing)
checkValidity()	Verifica si todos los campos son válidos	form.checkValidity()
setCustomValidity(msg)	Define un mensaje personalizado de error	campo.setCustomValidity("Formato incorrecto")
reportValidity()	Muestra visualmente los errores en el navegador	form.reportValidity()

Ejemplo con validación nativa y personalizada:

```
const email = document.querySelector("#email");

email.addEventListener("input", () => {
  if (!email.value.includes("@")) {
    email.setCustomValidity("Debe incluir un símbolo @");
  } else {
    email.setCustomValidity("");
  }
});
```

3. Validación dinámica con expresiones regulares (RegExp)

Para validar formatos específicos (número de cédula, correo, teléfono, contraseña, etc.), se utilizan expresiones regulares integradas con el DOM.

```
const telefono = document.querySelector("#telefono");
const msg = document.querySelector("#errorTelefono");
const regex = /^d{10}$/; // solo 10 dígitos

telefono.addEventListener("input", () => {
  if (!regex.test(telefono.value)) {
    msg.textContent = "El teléfono debe tener 10 dígitos.";
  } else {
    msg.textContent = "";
  }
});
```

Tabla 5. Ejemplos de expresiones regulares para validación

Tipo de dato	Ejemplo de expresión regular	Validación
Correo electrónico	<code>/^[^\s@]+@[^\s@]+\.[^\s@]+\$/</code>	Estructura básica de email
Contraseña segura	<code>/^(?=.*\d)(?=.*[A-Z])(?=.*[a-z]).{8,}\$/</code>	Letras, mayúsculas, número, 8+ caracteres
Cédula ecuatoriana (formato)	<code>/^\d{10}\$/</code>	Solo 10 dígitos numéricos
URL	<code>/^https?:\/\/[\w.-]+/</code>	Comienza con http o https

4. Feedback visual: clases dinámicas y atributos ARIA

Las validaciones deben ser claras y accesibles.

Para ello, el DOM API permite modificar las clases CSS y los atributos de accesibilidad (ARIA) al detectar errores o éxitos.

```
function mostrarEstado(campo, valido) {
  campo.setAttribute("aria-invalid", !valido);
  campo.classList.toggle("ok", valido);
  campo.classList.toggle("error", !valido);
}
```

CSS asociado:

```
input.ok { border: 2px solid green; }
input.error { border: 2px solid red; }
```

Esto no solo mejora la experiencia visual, sino que también comunica a los lectores de pantalla el estado del campo, garantizando accesibilidad.

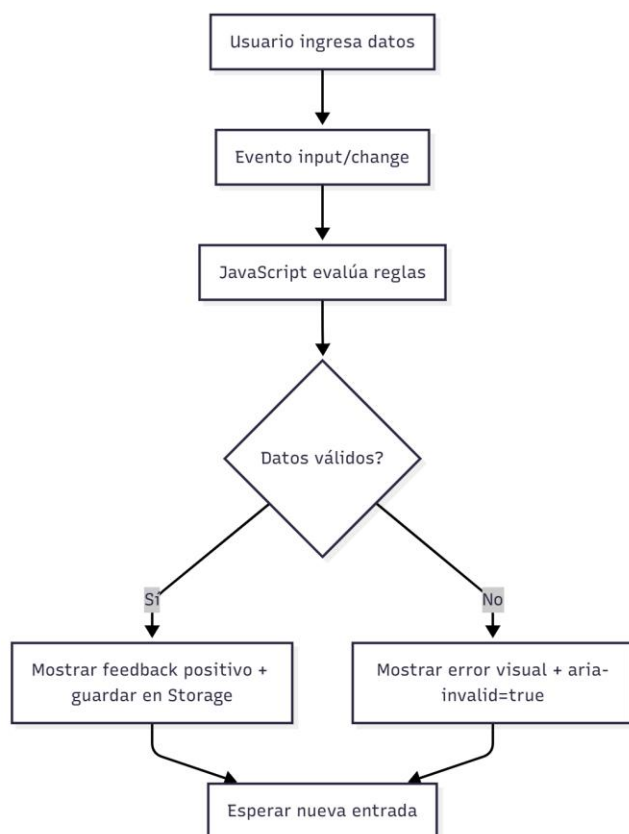


Imagen 4. Flujo de validación dinámica con DOM API

Fuente: Elaboración propia (2025)

5. Integración con Web Storage (localStorage y sessionStorage)

El navegador incluye dos sistemas de almacenamiento clave para persistir datos, por ejemplo de validación:

Tabla 6. Web Storage

API	Duración	Acceso	Ejemplo de uso
localStorage	Permanente (hasta ser borrado)	Todas las pestañas del mismo origen	Recordar nombre o preferencias del usuario
sessionStorage	Temporal (mientras dure la pestaña)	Solo sesión actual	Guardar estado temporal del formulario

Ambas funcionan con pares **clave–valor** y solo almacenan texto (*string*), por lo que es común usar `JSON.stringify()` y `JSON.parse()`.



Ejemplo práctico: persistencia de formulario

```
<form id="perfil">
  <input id="nombre" placeholder="Nombre" />
  <input id="correo" type="email" placeholder="Correo" />
  <button id="guardar">Guardar</button>
</form>

<script>
  const nombre = document.querySelector("#nombre");
  const correo = document.querySelector("#correo");

  // 1. Cargar datos previos
  nombre.value = localStorage.getItem("nombre") || "";
  correo.value = localStorage.getItem("correo") || "";

  // 2. Guardar al hacer clic
  document.querySelector("#guardar").addEventListener("click", (e) => {
    e.preventDefault();
    localStorage.setItem("nombre", nombre.value);
    localStorage.setItem("correo", correo.value);
    alert("Datos guardados en localStorage");
  });
</script>
```

Resultado: los datos permanecen incluso al cerrar el navegador.

6. Validación con persistencia temporal (sessionStorage)

Ideal cuando se requiere mantener información solo durante la sesión (por ejemplo, un formulario de contacto).

```
const form = document.querySelector("#contacto");
const campo = document.querySelector("#mensaje");

campo.value = sessionStorage.getItem("mensaje") || "";

campo.addEventListener("input", () => {
  sessionStorage.setItem("mensaje", campo.value);
});
```

Esto evita perder el contenido si el usuario recarga accidentalmente la página.

7. Ejemplo integrador: validación + almacenamiento

El siguiente fragmento combina todo lo aprendido:

- Validación en tiempo real.
- Persistencia local.
- Retroalimentación visual y accesible.



```
<form id="formulario">
  <label>Correo:</label>
  <input id="correo" type="email" aria-describedby="errorCorreo" />
  <small id="errorCorreo"></small>
  <button>Guardar</button>
</form>

<script>
  const correo = document.querySelector("#correo");
  const errorCorreo = document.querySelector("#errorCorreo");

  // Cargar valor previo
  correo.value = localStorage.getItem("correo") || "";

  correo.addEventListener("input", () => {
    const valido = /^[^@\s]+@[^\s]+\.[^\s]+\$/i.test(correo.value);
    correo.setAttribute("aria-invalid", !valido);
    errorCorreo.textContent = valido ? "" : "Formato de correo inválido";
    correo.classList.toggle("error", !valido);

    // Guardar valor solo si es válido
    if (valido) localStorage.setItem("correo", correo.value);
  });
</script>
```

Tabla 7. Comparativa de mecanismos de almacenamiento en navegador

Tecnología	Capacidad aprox.	Persistencia	Tipo de datos	Seguridad
localStorage	5–10 MB	Permanente	Texto (string)	Accesible por JS
sessionStorage	5 MB	Hasta cerrar pestaña	Texto (string)	Accesible por JS
cookies	4 KB	Definida por servidor o script	Texto	Se envían en cada petición
IndexedDB	50 MB o más	Permanente	Objetos complejos	Más segura y estructurada

8. Mejora de rendimiento con DOM API

La validación dinámica puede degradar el rendimiento si se manipula el DOM con demasiada frecuencia. Para optimizar:

1. Usar DocumentFragment para actualizaciones múltiples.
2. Evitar repintar nodos en cada pulsación (usar debounce).
3. Reutilizar referencias a los elementos (const input = ...).
4. Actualizar solo lo necesario (diferir tareas con requestAnimationFrame()).



```
let timeout;
input.addEventListener("input", () => {
  clearTimeout(timeout);
  timeout = setTimeout(() => validar(), 300); // ejecuta tras 300 ms sin escribir
});
```

9. Conexión con almacenamiento avanzado: IndexedDB y Cookies (vista previa)

Aunque `localStorage` es útil, sus límites se superan con IndexedDB, una base de datos interna del navegador que permite almacenar objetos estructurados y realizar búsquedas con índices. También pueden utilizarse cookies para guardar sesiones ligeras o tokens de autenticación, aunque se recomienda combinarlas con políticas de seguridad modernas (`SameSite`, `Secure`). Estos temas se profundizarán en el punto 5.4, donde se integrará almacenamiento y accesibilidad.

10. Conclusión

La validación dinámica mediante el DOM API transforma los formularios tradicionales en sistemas inteligentes que responden en tiempo real a las acciones del usuario. Combinada con las capacidades de almacenamiento local (`localStorage` y `sessionStorage`), permite crear aplicaciones más usables, accesibles y persistentes, sin depender del servidor.

5.3. Fetch API y consumo de datos

La capacidad de una aplicación para obtener y enviar información a un servidor es esencial en el desarrollo de plataformas web modernas. La Fetch API de JavaScript representa la evolución natural del antiguo objeto `XMLHttpRequest`, ofreciendo una interfaz promesificada, más clara y flexible para trabajar con solicitudes HTTP asíncronas.

A través de `fetch()`, los desarrolladores pueden conectar sus aplicaciones con APIs REST, servicios externos o incluso recursos locales, sin bloquear el flujo del programa. Combinada con las herramientas de almacenamiento del navegador —como IndexedDB y cookies—, esta API permite construir aplicaciones capaces de sincronizar y conservar datos incluso sin conexión a Internet, sentando las bases del concepto de aplicaciones web progresivas (PWA).

1. ¿Qué es Fetch API?

La Fetch API es una interfaz moderna basada en promesas que permite realizar solicitudes HTTP para recuperar (GET) o enviar (POST, PUT, DELETE) información.

Sintaxis básica

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error("Error al cargar:", error));
```

- `fetch()` devuelve una promesa.
- El método `.json()` transforma la respuesta en un objeto JavaScript.
- `catch()` gestiona los posibles errores de red o de parseo.

Ventaja principal: la sintaxis es más legible, clara y compatible con `async/await`.

2. Fetch API con `async/await`

El mismo código puede escribirse de forma más secuencial y limpia usando `async/await`:

```
async function cargarUsuarios() {
  try {
    const respuesta = await fetch("https://jsonplaceholder.typicode.com/users");
    const datos = await respuesta.json();
    console.table(datos);
  } catch (error) {
    console.error("Error:", error.message);
  }
}
cargarUsuarios();
```

Esta estructura permite manejar los errores mediante `try...catch`, integrándose fácilmente con las técnicas de manejo de errores vistas en la **clase 4**.

3. Métodos y opciones en Fetch

`fetch()` admite un segundo parámetro con opciones de configuración como método, encabezados o cuerpo:

```
fetch("https://api.example.com/login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    usuario: "damian",
    clave: "12345"
  })
})
  .then(r => r.json())
  .then(data => console.log("Respuesta:", data))
  .catch(err => console.error(err));
```

Tabla 8. Estructura de una petición fetch

Opción	Descripción	Ejemplo
method	Tipo de solicitud HTTP	"GET", "POST", "PUT", "DELETE"
headers	Encabezados personalizados	{ "Content-Type": "application/json" }
body	Cuerpo de la petición (en POST o PUT)	JSON.stringify(objeto)
mode	Controla el CORS	"cors", "no-cors", "same-origin"
credentials	Envío de cookies	"include", "same-origin"

4. Flujo completo de solicitud con control de errores

```

async function obtenerPublicaciones() {
  const url = "https://jsonplaceholder.typicode.com/posts";
  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(`Error HTTP ${res.status}`);
    const data = await res.json();
    data.slice(0, 3).forEach(p => console.log(p.title));
  } catch (e) {
    console.error("Error al obtener publicaciones:", e.message);
  } finally {
    console.log("Solicitud finalizada");
  }
}
obtenerPublicaciones();

```

Explicación:

- Se verifica el estado HTTP con `res.ok`.
- Se limita la salida a 3 registros (`slice(0, 3)`).
- El bloque `finally` garantiza la ejecución posterior.

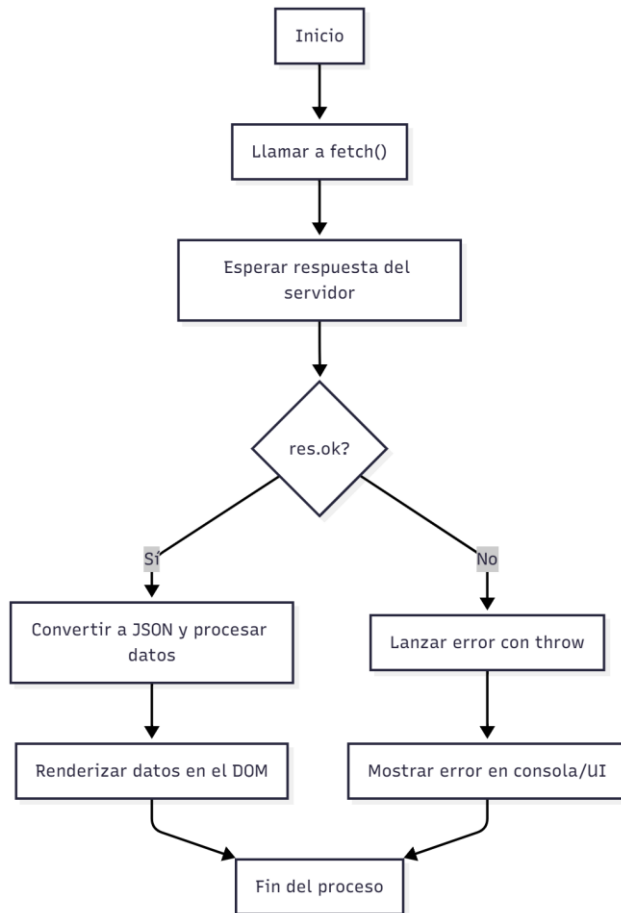


Imagen 5. Flujo de ejecución Fetch API con async/await

Fuente: Elaboración propia (2025)

5. Consumo dinámico de datos en el DOM

El siguiente ejemplo muestra cómo inyectar los datos obtenidos directamente en la interfaz:

```

<ul id="usuarios"></ul>

<script>
async function mostrarUsuarios() {
  const lista = document.querySelector("#usuarios");
  lista.innerHTML = "<li>Cargando...</li>";
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    const datos = await res.json();
    lista.innerHTML = datos.map(u => `<li>${u.name} - ${u.email}</li>`).join("");
  } catch {
    lista.innerHTML = "<li>Error al cargar usuarios.</li>";
  }
}
mostrarUsuarios();
</script>
  
```

Resultado: Los usuarios se listan automáticamente en el DOM al cargarse la página.

6. Almacenamiento de datos obtenidos con IndexedDB

El IndexedDB API permite almacenar grandes volúmenes de datos estructurados (objetos, listas, JSON) en el navegador. A diferencia de `localStorage`, que solo guarda texto, IndexedDB admite índices, búsquedas y transacciones, siendo ideal para aplicaciones offline.

Estructura básica:

```
// 1. Abrir base de datos
const solicitud = indexedDB.open("MiBase", 1);

solicitud.onupgradeneeded = (e) => {
  const db = e.target.result;
  db.createObjectStore("usuarios", { keyPath: "id" });
};

// 2. Insertar datos
solicitud.onsuccess = () => {
  const db = solicitud.result;
  const trans = db.transaction("usuarios", "readwrite");
  const store = trans.objectStore("usuarios");

  fetch("https://jsonplaceholder.typicode.com/users")
    .then(r => r.json())
    .then(datos => datos.forEach(u => store.put(u)));
};
```

Ventaja: Los datos quedan almacenados incluso sin conexión. En la siguiente carga, la aplicación puede consultarlos directamente sin recurrir al servidor.

Tabla 9. Comparativa entre mecanismos de almacenamiento

API	Capacidad	Tipo de datos	Persistencia	Acceso JS	Ideal para
<code>localStorage</code>	5-10 MB	Texto	Permanente	Fácil	Preferencias, formularios
<code>sessionStorage</code>	5 MB	Texto	Temporal	Fácil	Sesiones activas
IndexedDB	50+ MB	Objetos complejos	Permanente	Avanzado	Datos offline, catálogos
<code>cookies</code>	4 KB	Texto	Definida	Moderado	Sesiones, tokens

7. Sincronización entre Fetch y IndexedDB

Para lograr una sincronización progresiva:

1. Intentar obtener datos del servidor (fetch).
2. Si falla, recuperar los datos guardados en IndexedDB.
3. Mostrar los resultados en la interfaz, garantizando continuidad.



```
async function obtenerDatosOffline() {
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!res.ok) throw new Error("Error HTTP");
    const data = await res.json();
    guardarEnIndexedDB(data);
    renderizar(data);
  } catch {
    console.warn("Servidor inaccesible, usando datos locales...");
    const dataLocal = await leerDesdeIndexedDB();
    renderizar(dataLocal);
  }
}
```

Este patrón es común en Progressive Web Apps (PWA) para mantener la funcionalidad aun sin conexión.

8. Uso de cookies para autenticación y preferencias

Las cookies son fragmentos pequeños de información que el navegador almacena y envía automáticamente al servidor con cada solicitud HTTP. Sirven para mantener sesiones, autenticaciones o recordar configuraciones del usuario.

```
// Crear cookie
document.cookie = "usuario=Damián; max-age=3600; path="/";

// Leer cookies
console.log(document.cookie);

// Borrar cookie
document.cookie = "usuario=; max-age=0; path="/";
```

Buenas prácticas:

- Usar Secure para HTTPS.
- Usar SameSite=Strict para prevenir ataques CSRF.
- No guardar información sensible (como contraseñas).

Ejemplo seguro:

```
Set-Cookie: sesion=abc123; Secure; HttpOnly; SameSite=Strict
```

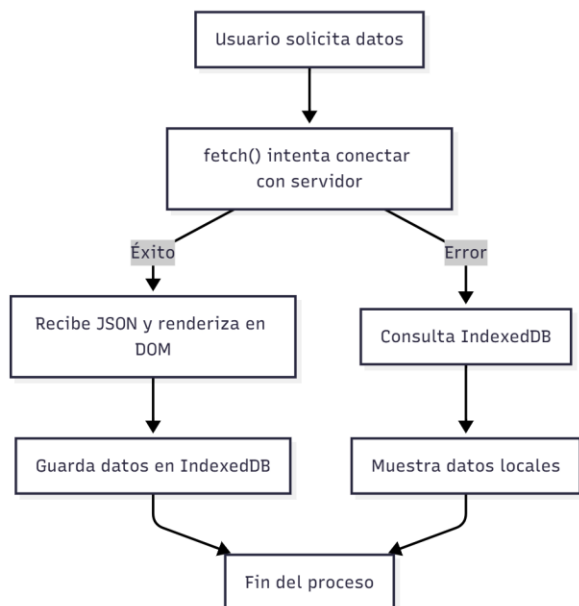


Imagen 6. Flujo de almacenamiento y consumo con Fetch, IndexedDB y cookies

Fuente: Elaboración propia (2025)



9. Ejemplo integrador: Fetch + IndexedDB + cookies

```
<button id="actualizar">Actualizar usuarios</button>
<ul id="usuarios"></ul>
<script>
  const DB_NAME = "UsuariosDB";
  const STORE = "usuarios";
  let db;

  // Inicializar base de datos
  const req = indexedDB.open(DB_NAME, 1);
  req.onupgradeneeded = e => e.target.result.createObjectStore(STORE, { keyPath: "id" });
  req.onsuccess = e => db = e.target.result;

  // Guardar en IndexedDB
  function guardar(data) {
    const tx = db.transaction(STORE, "readwrite");
    const store = tx.objectStore(STORE);
    data.forEach(u => store.put(u));
  }

  // Leer desde IndexedDB
  function leer(callback) {
    const tx = db.transaction(STORE, "readonly");
    const store = tx.objectStore(STORE);
    const req = store.getAll();
    req.onsuccess = () => callback(req.result);
  }

  // Renderizar en DOM
  function render(users) {
    document.querySelector("#usuarios").innerHTML =
      users.map(u => `<li>${u.name} (${u.email})</li>`).join("");
  }

  // Función principal
  document.querySelector("#actualizar").addEventListener("click", async () => {
    try {
      const res = await fetch("https://jsonplaceholder.typicode.com/users");
      const data = await res.json();
      guardar(data);
      document.cookie = "ultimaActualizacion=" + new Date().toISOString();
      render(data);
    } catch {
      leer(render);
    }
  });
</script>
```

Resultado:

- Si hay conexión: los datos se descargan, guardan y muestran.
- Si no hay conexión: se cargan los datos almacenados previamente.
- Se registra la última actualización en una cookie.

10. Conclusión

La Fetch API marca un hito en la programación web moderna, al ofrecer una forma simple y elegante de comunicarse con servidores externos de manera asíncrona. Cuando se combina con IndexedDB y cookies, las aplicaciones pueden mantener su funcionalidad incluso sin conexión, sincronizando datos automáticamente cuando el servicio se restablece.



Dominar este ecosistema —Fetch + Storage + IndexedDB + cookies— permite crear experiencias fluidas, seguras y resilientes, base de toda arquitectura de plataformas inteligentes y aplicaciones progresivas (PWA).

Estos conocimientos serán esenciales para el cierre de la semana, donde se integrarán con

5.4. Accesibilidad con atributos ARIA

La accesibilidad es un componente esencial del desarrollo en plataformas modernas. Implica garantizar que todas las personas, independientemente de sus capacidades físicas, cognitivas o tecnológicas, puedan interactuar con una aplicación web de forma plena y eficiente.

En la práctica, esto significa diseñar sistemas que puedan ser comprendidos y controlados tanto por un usuario vidente con un mouse, como por uno que navega mediante un lector de pantalla, un teclado o una interfaz táctil.

En este contexto, la especificación WAI-ARIA (Web Accessibility Initiative – Accessible Rich Internet Applications) se convierte en una herramienta fundamental. ARIA complementa el HTML y el DOM mediante un conjunto de atributos semánticos que comunican información adicional sobre el rol, el estado y las propiedades de los elementos.

Cuando se integran con las API de JavaScript, los eventos y los sistemas de validación, permiten crear interfaces dinámicas accesibles, sin sacrificar interactividad ni usabilidad.

1. Principios de accesibilidad: perceivable, operable, understandable, robust (POUR)

Los principios definidos por la WCAG 2.1 (Web Content Accessibility Guidelines) establecen el marco general de la accesibilidad digital:

Tabla 10. Principios de accesibilidad POUR

Principio	Descripción	Ejemplo práctico
Perceptible	La información debe ser visible o audible.	Contraste de color suficiente, texto alternativo en imágenes.
Operable	El contenido debe ser navegable por teclado.	Botones accesibles y foco visible.
Comprensible	Las instrucciones y mensajes deben ser claros.	Validaciones con mensajes simples y descriptivos.
Robusto	Compatible con distintas tecnologías asistivas.	Uso correcto de roles ARIA y etiquetas semánticas.

En el ámbito de esta clase, estos principios se aplican al DOM dinámico, las validaciones en tiempo real y la interacción con APIs, garantizando que el contenido siga siendo accesible incluso cuando cambia mediante JavaScript.



2. Qué es WAI-ARIA y cuándo se utiliza

Los **atributos ARIA** no reemplazan el HTML estándar, sino que complementan su semántica cuando los elementos nativos no comunican adecuadamente su propósito o estado.

Por ejemplo, un botón creado con un `<div>` necesita un rol ARIA para ser identificado correctamente por un lector de pantalla:

```
<div role="button" tabindex="0" aria-pressed="false">Aceptar</div>
```

- `role="button"` indica el propósito.
- `tabindex="0"` permite la navegación con teclado.
- `aria-pressed` comunica el estado (presionado o no).

Regla clave: Usar ARIA solo cuando el HTML semántico no sea suficiente. Un `<button>` bien implementado es preferible a un `<div role="button">`.

3. Atributos ARIA más comunes

Tabla 11. Atributos ARIA

Categoría	Atributo	Descripción	Ejemplo
Roles	<code>role</code>	Define el propósito del elemento	<code>role="dialog", role="alert"</code>
Estados	<code>aria-checked</code> , <code>aria-expanded</code> , <code>aria-hidden</code>	Indican estado actual	<code>aria-expanded="true"</code>
Propiedades	<code>aria-label</code> , <code>aria-labelledby</code> , <code>aria-describedby</code>	Añaden descripción o contexto	<code>aria-describedby="msgError"</code>
Interactividad	<code>aria-live</code> , <code>aria-busy</code>	Define regiones dinámicas	<code>aria-live="polite"</code>
Validación	<code>aria-invalid</code> , <code>aria-required</code>	Indican error o requerimiento	<code>aria-invalid="true"</code>

4. Validaciones accesibles en formularios dinámicos

En validaciones construidas con JavaScript (como las del punto 5.2), es fundamental informar a las tecnologías asistivas del estado de los campos.

Esto se logra mediante `aria-invalid` y `aria-describedby`.

```
<label for="correo">Correo electrónico</label>
<input id="correo" aria-describedby="errorCorreo" aria-invalid="false" />
<small id="errorCorreo" class="error" aria-live="assertive"></small>

<script>
const correo = document.querySelector("#correo");
const msg = document.querySelector("#errorCorreo");
```

```
correo.addEventListener("input", () => {
  const valido = /^[^@\s]+@[^\s]+\.[^\s]+\$/ .test(correo.value);
  correo.setAttribute("aria-invalid", !valido);
  msg.textContent = valido ? "" : "Formato de correo inválido";
});
</script>
```

Qué sucede aquí:

- El lector de pantalla anuncia el mensaje de error cuando aparece.
- `aria-invalid` pasa de `false` a `true`, comunicando el estado.
- El atributo `aria-live="assertive"` indica que el cambio debe leerse inmediatamente.

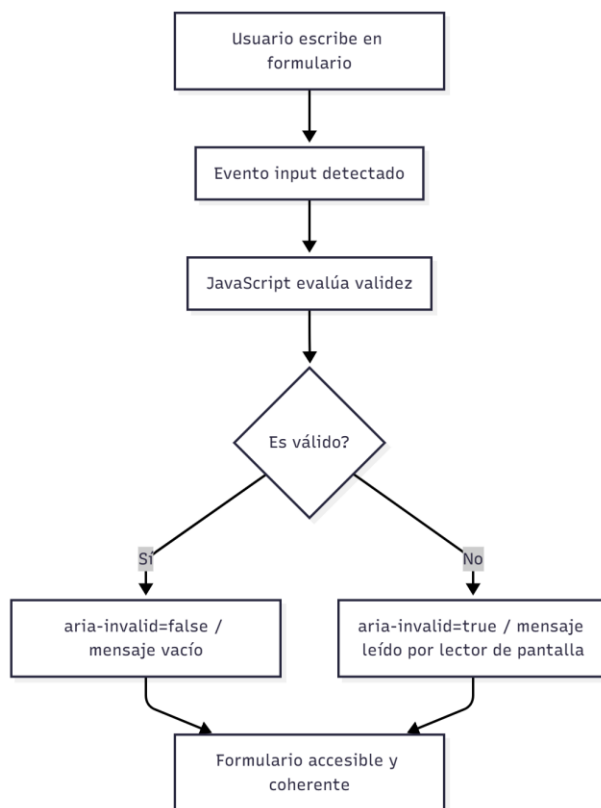


Imagen 7. Flujo de validación accesible con ARIA

Fuente: Elaboración propia (2025)

5. Atributos ARIA para contenido dinámico

Cuando se modifican secciones del DOM (por ejemplo, con Fetch API), es importante que el lector de pantalla detecte el cambio.

Para ello se usa `aria-live`, que define regiones “vivas”.



```
<div id="estado" aria-live="polite"></div>

<script>
async function cargarDatos() {
  const estado = document.getElementById("estado");
  estado.textContent = "Cargando...";
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users/1");
    const data = await res.json();
    estado.textContent = `Usuario cargado: ${data.name}`;
  } catch {
    estado.textContent = "Error al cargar información.";
  }
}
</script>
```

El lector de pantalla detectará automáticamente los cambios en el texto de #estado, anunciando la actualización sin que el usuario deba volver a enfocar el elemento.

6. Ejemplo: interfaz de lista accesible

```
<ul id="usuarios" role="list" aria-live="polite"></ul>

<script>
async function mostrarUsuarios() {
  const lista = document.querySelector("#usuarios");
  lista.textContent = "Cargando usuarios...";
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    const datos = await res.json();
    lista.innerHTML = datos.map(u =>
      `<li role="listitem">${u.name} - ${u.email}</li>`).join("");
  } catch {
    lista.textContent = "No se pudieron cargar los usuarios.";
  }
}
mostrarUsuarios();
</script>
```

Con role="list" y aria-live="polite", los usuarios con lector de pantalla sabrán que el contenido fue actualizado dinámicamente.

7. Integración con teclas y accesibilidad por teclado

Toda aplicación debe ser navegable sin mouse.

El atributo tabindex permite definir el orden de navegación:

```
<div tabindex="0" role="button">Aceptar</div>
<div tabindex="0" role="button">Cancelar</div>
```

También puede capturarse la tecla Enter para activar acciones:

```
document.addEventListener("keydown", (e) => {
  if (e.key === "Enter" && e.target.role === "button") {
    e.target.click();
  }
});
```



8. Uso combinado de ARIA y almacenamiento

Cuando una aplicación guarda datos (por ejemplo, en `localStorage` o `IndexedDB`), puede informar accesiblemente al usuario del resultado:

```
<p id="mensaje" aria-live="assertive"></p>

<script>
function guardarPreferencia() {
  localStorage.setItem("tema", "oscuro");
  document.querySelector("#mensaje").textContent =
    "Preferencia guardada: modo oscuro activado.";
}
</script>
```

De esta forma, los usuarios que no ven la pantalla escucharán la confirmación del cambio gracias a `aria-live`.

9. Accesibilidad y Fetch API: mensajes de carga

Cuando se cargan datos de forma asíncrona (por ejemplo, con `Fetch`), la accesibilidad debe mantenerse.

El atributo `aria-busy` indica que un elemento está ocupado:

```
<div id="panel" aria-busy="false" aria-live="polite"></div>

<script>
async function actualizarPanel() {
  const panel = document.querySelector("#panel");
  panel.setAttribute("aria-busy", "true");
  panel.textContent = "Cargando información...";
  const res = await fetch("https://jsonplaceholder.typicode.com/todos/1");
  const data = await res.json();
  panel.textContent = data.title;
  panel.setAttribute("aria-busy", "false");
}
</script>
```

Cuando `aria-busy="true"`, los lectores de pantalla saben que la región está temporalmente inactiva y evitarán leerla hasta que se complete la operación.

Tabla 12. Atributos ARIA esenciales para interfaces dinámicas

Atributo	Propósito	Uso típico
<code>aria-live</code>	Avisar cambios en regiones dinámicas	"Cargando... / Datos actualizados"
<code>aria-busy</code>	Indicar procesamiento en curso	Paneles de contenido o formularios
<code>aria-invalid</code>	Señalar error en un campo	Formularios con validación
<code>aria-describedby</code>	Relacionar campo con mensaje de ayuda o error	<code>input</code> y <code>small</code>
<code>aria-hidden</code>	Ocultar elementos a lectores de pantalla	Contenido visual sin relevancia



aria-expanded	Mostrar si una sección está desplegada	Menús o acordeones
role="alert"	Comunicar mensajes urgentes	Notificaciones o errores críticos

10. Ejemplo integrador: formulario accesible y persistente

```
<form id="registro">
  <label for="nombre">Nombre</label>
  <input id="nombre" aria-describedby="errNombre" aria-required="true" />
  <small id="errNombre" aria-live="assertive"></small>

  <label for="correo">Correo</label>
  <input id="correo" type="email" aria-describedby="errCorreo" />
  <small id="errCorreo" aria-live="assertive"></small>

  <button>Guardar</button>
</form>

<script>
const nombre = document.querySelector("#nombre");
const correo = document.querySelector("#correo");

[nombre, correo].forEach(campo => {
  const idError = campo.getAttribute("aria-describedby");
  const msg = document.getElementById(idError);

  campo.value = localStorage.getItem(campo.id) || "";

  campo.addEventListener("input", () => {
    const valido = campo.value.trim().length > 2;
    campo.setAttribute("aria-invalid", !valido);
    msg.textContent = valido ? "" : "Debe contener al menos 3 caracteres";
    if (valido) localStorage.setItem(campo.id, campo.value);
  });
});
</script>
```

Características:

- Validación en tiempo real con feedback audible (aria-live).
- Persistencia en localStorage.
- Campos accesibles mediante teclado y lectores de pantalla.

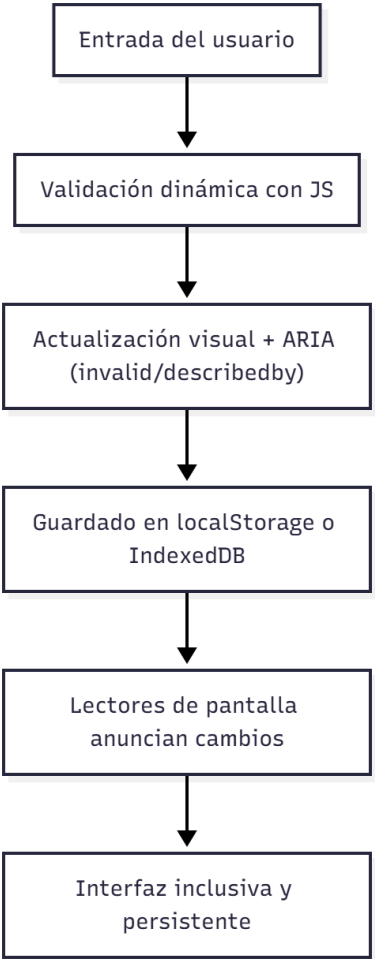


Imagen 8. Relación entre accesibilidad, validación y almacenamiento

Fuente: Elaboración propia (2025)

11. Buenas prácticas generales de accesibilidad

Tabla 12. Buenas prácticas de accesibilidad

Categoría	Recomendación	Beneficio
Semántica	Usar etiquetas HTML correctas antes de aplicar ARIA	Mejora compatibilidad
Color y contraste	Asegurar contraste mínimo de 4.5:1	Legibilidad en todos los dispositivos
Foco y navegación	Mantener foco visible en todos los elementos interactivos	Navegación por teclado fluida
Mensajes	Evitar solo color para indicar estado	Inclusión de usuarios con daltonismo
Retroalimentación	Usar <code>aria-live</code> para mensajes dinámicos	Comunicación instantánea a lectores



Pruebas	Validar con herramientas como <i>axe</i> <i>DevTools</i> o <i>Lighthouse</i>	Verificación automatizada
---------	--	---------------------------

12. Conclusión

La accesibilidad con atributos ARIA no es un complemento opcional, sino una obligación ética y técnica en el diseño de plataformas digitales.

Implementar estos atributos junto con el DOM API, Fetch, eventos, y los sistemas de almacenamiento permite desarrollar aplicaciones inclusivas, reactivas y coherentes, en las que cada interacción es perceptible, operable, comprensible y robusta.

Al dominar estos conceptos, el estudiante estará en capacidad de construir interfaces web inteligentes, asincrónicas y accesibles, cumpliendo los estándares internacionales de calidad y usabilidad (WCAG 2.1 y WAI-ARIA 1.2).

Referencias citadas en la Clase 5.

- Mozilla Developer Network (MDN). (2025). ARIA: Accessible Rich Internet Applications. Recuperado de <https://developer.mozilla.org/es/docs/Web/Accessibility/ARIA>
- World Wide Web Consortium (W3C). (2018). Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation.
- World Wide Web Consortium (W3C). (2021). WAI-ARIA 1.2 Specification.
- Paciello Group. (2024). Practical Accessibility for Web Developers.
- Duckett, J. (2022). JavaScript and jQuery: Interactive Front-End Web Development. Wiley Publishing.
- Powell, T. (2023). HTML & CSS: The Complete Reference (5th Edition). McGraw-Hill Education.

Definición de términos

- **Accesibilidad web (Web Accessibility):** Conjunto de prácticas, estándares y tecnologías que garantizan que las personas con discapacidad puedan percibir, entender, navegar e interactuar con los sitios web de manera efectiva, independientemente de sus capacidades o dispositivos utilizados.
- **WAI-ARIA (Web Accessibility Initiative – Accessible Rich Internet Applications):** Especificación desarrollada por el W3C que amplía la semántica de HTML mediante atributos especiales (aria-*) para describir roles, estados y propiedades de los elementos dinámicos, facilitando la comprensión de las interfaces por parte de tecnologías asistivas como lectores de pantalla.

- Recursos de profundización

Recurso: *Profundizacion1_clase5.docx*

ACTIVIDAD DE EVALUACIÓN:



INSTRUCCIONES	Responde las siguientes preguntas de opción múltiple relacionadas con los temas abordados en la Clase.				
	Selecciona la alternativa correcta en cada caso. Al finalizar, revisa la retroalimentación para reforzar tu comprensión.				
Titulo	Manipulación del DOM y eventos				
Dificultad		Baja	X	Media	Alta
RDAs Evaluados:	X	RDA 1			
		RDA 2			
		RDA 3			
Secciones Evaluadas		Clase: 5	Manipulación del DOM y eventos		
PREGUNTA 1	¿Cuál de los siguientes atributos ARIA se utiliza para informar a los lectores de pantalla que una región está siendo actualizada dinámicamente?				
CORRECTA	aria-live				
Incorrecta	aria-busy				
Incorrecta	aria-invalid				
Incorrecta	aria-hidden				
Retroalimentación de las respuestas	aria-live permite que los lectores de pantalla anuncien automáticamente los cambios en el contenido dinámico, como mensajes de carga o actualizaciones de datos.				
PREGUNTA 2	¿Cuál es la función de aria-invalid="true" en un campo de formulario?				



CORRECTA	Indica que el campo contiene un error de validación.
Incorrecta	Muestra un mensaje de error automáticamente.
Incorrecta	Marca el campo como obligatorio.
Incorrecta	Oculto el campo al lector de pantalla.
Retroalimentación de las respuestas	El atributo <code>aria-invalid</code> comunica a las tecnologías asistivas que un campo tiene un valor inválido, sin modificar su comportamiento visual directamente.
PREGUNTA 3	En el contexto de accesibilidad, ¿cuándo es recomendable usar atributos ARIA?
CORRECTA	Solo cuando el HTML semántico no es suficiente para expresar el rol o el estado de un elemento.
Incorrecta	Siempre, en lugar de etiquetas HTML estándar.
Incorrecta	Solo en componentes de frameworks modernos.
Incorrecta	Solo en aplicaciones móviles.
Retroalimentación de las respuestas	ARIA complementa, pero no sustituye, la semántica HTML. Los elementos nativos (<code><button></code> , <code><input></code> , <code><nav></code>) ya contienen roles accesibles integrados.
PREGUNTA 4	¿Qué atributo ARIA puede aplicarse a una región para indicar que se encuentra en proceso de carga?
CORRECTA	<code>aria-busy="true"</code>
Incorrecta	<code>aria-live="assertive"</code>
Incorrecta	<code>aria-hidden="true"</code>
Incorrecta	<code>aria-expanded="false"</code>



Retroalimentación de las respuestas	aria-busy comunica que un componente está temporalmente ocupado (por ejemplo, cargando datos mediante Fetch), y que los lectores de pantalla deben esperar hasta que el proceso finalice para leer el contenido.
-------------------------------------	--

Enlaces externos recomendados

Enlace 1

- **Título del enlace relacionado:**
Guía oficial de ARIA – W3C
- **Descripción del enlace relacionado:**
Documento técnico completo del W3C que describe los roles, estados y propiedades de la especificación WAI-ARIA 1.2, con ejemplos prácticos de implementación en aplicaciones web dinámicas.
- **Enlace:**
<https://www.w3.org/TR/wai-aria-1.2/>

Enlace 2

- **Título del enlace relacionado:**
Pruebas de accesibilidad en el navegador con axe DevTools
- **Descripción del enlace relacionado:**
Extensión gratuita para navegadores que permite auditar la accesibilidad de un sitio web en tiempo real, detectando errores de contraste, etiquetas ausentes, roles mal definidos o regiones no anunciadas por ARIA.
- **Enlace:**
<https://www.deque.com/axe/devtools/>